CPT204 2022-2023 F S2 - Suggested Answers

Note: Question 1 short-answer section is removed in the publishable paper, so only Q2-Q3 can be answered from the available file.

Question 2 Coding 1 - addFirst(T item)

Idea:
- Before adding, count whether item already occurs more than once in the current deque.
- If it already occurs at least once and adding it would make it occur more than once, throw IllegalArgumentException.
- Otherwise add it to the front.

Typical answer pattern:

```
public void addFirst(T item) {
    int count = 0;
    for (T x : this) {
        if (item == null ? x == null : item.equals(x)) {
            count++;
        }
    }
    if (count >= 1) {
        throw new IllegalArgumentException();
    }
    super.addFirst(item);
}
```

If the wording is interpreted as "already occurs more than once before adding", change the condition to count > 1. The given test case expects the second "a" to fail, so count >= 1 is the useful implementation.

Question 3 Coding 2 - equals(Object that)

Requirement:
- same type / compatible ARDeque
- same size
- at most 1 different item at the same position

Typical answer pattern:

```
@Override
public boolean equals(Object that) {
    if (this == that) return true;
    if (!(that instanceof ARDeque<?>)) return false;

    ARDeque<?> other = (ARDeque<?>) that;
    if (this.size() != other.size()) return false;

    Iterator<T> it1 = this.iterator();
    Iterator<?> it2 = other.iterator();
    int different = 0;

    while (it1.hasNext() && it2.hasNext()) {
        T a = it1.next();
        Object b = it2.next();
        if (!(a == null ? b == null : a.equals(b))) {
            different++;
            if (different > 1) return false;
        }
    }
```

```
    return true;
}
```